

深圳大学实验报告

课程名称： 编译原理

实验项目名称： 词法分析程序设计

学院： 计算机与软件学院

专业： 软件工程

指导教师： 王祎乐

报告人： 学号： 班级：

实验时间： 2026 年 4 月 24 日

实验报告提交时间： 2026 年 5 月 8 日

教务处制

实验目的与要求:

目的：通过实验，加深对词法分析技术的理解。

要求:

第一部分：词法分析的理解

词法规则：

类别编号	单词类别	示例	说明
1	关键字	int,float, program, const, def, begin, end, call, let, if, then, else, fi, while, do, endwh, return, and, or	关键字 区分大小写。关键字仅包含示例中的
2	标识符	变量名、函数名等（如 myVar, Test123）	由 字母开头,可包含 字母和数字, 最长 32 个字符, 不区分大小写。
3	整数	0, 234, 10000	无符号十进制整数,首位不能为0(除非是单独的 0),最大值 16 位。
4	小数 (浮点数)	0.7, 12.43, 9.0	由 数字+小数点+数字 组成, 必须有小数点, 不考虑指数形式。
5	分隔符	();,	包括 括号、分号、逗号。
6	四则运算符	+ - * /	加法、减法、乘法、除法。
7	比较运算符	==, <, <=, >, >=, <>	关系运算符, 用于比较大小。
8	赋值运算符	=	单个 = 代表赋值。

输入文件：使用该词法规则编写的源码文件（共两个：source_correct.txt、source_error.txt）；以及空白、注释等。

输出：每个单词需要输出：（单词类型、单词本身、行号、列号）。如遇到错误，则单词本身为 ERROR，单词类型参考 Token.h 中的定义。每一个单词信息单独占一行，具体输出格式可参见后续的样例。

例如，对于如下的程序片段：

```
int  a;  
let a = 30;
```

要求输出如下所示：

```
(base) wangweiqin@wangweiqindeMac-mini ans % ./main test.txt
(1, int, 1, 1)
(2, a, 1, 6)
(5, ;, 1, 7)
(1, let, 2, 1)
(2, a, 2, 5)
(8, =, 2, 7)
(3, 30, 2, 9)
(5, ;, 2, 11)
```

注：代码整体框架已经写好，仅要求填写部分代码或者某些函数实现（共计十处，详见 [README.pdf or README.md](#)），可能会有些复杂，但是请务必看完以加深理解。在面对复杂代码时，鼓励使用大模型工具进行代码理解。

第二部分：程序的测试

在完成第一部分后，写一点简单的代码，输入到上面的词法分析器中，并给出执行词法分析后的输出。

方法、步骤：

1. 词法分析的大致流程是，结合课程内容？

词法分析是编译过程的第一个阶段，其主要任务是将输入的字符流转换为词法单元序列。以下是词法分析的大致流程：

1.1 初始化

（1）读取输入文件：打开待分析的源文件，准备逐字符读取文件内容。在提供的代码中，Lexer 类的构造函数 `Lexer::Lexer(const string& filename)` 负责打开文件，并初始化一些必要的变量，如行号 `line`、列号 `col` 和文件结束标志 `eof`。

（2）初始化关键字列表：将编程语言中的关键字存储在一个数据结构中，方便后续判断某个标识符是否为关键字。在代码里，关键字被存储在 `keywords` 向量中。

1.2 字符读取与处理

（1）跳过空白字符和注释：在读取字符时，需要跳过源文件中的空白字符（如空格、制表符、换行符）和注释，因为它们通常不影响程序的语义。代码中的 `skipWhitespace()` 函数用于跳过空白字符，`skipComment()` 函数用于跳过单行注释和多行注释。

（2）逐字符读取：使用 `advance()` 函数从文件中读取下一个字符，并更新当前的行号和列号。当读取到文件末尾时，将 `eof` 标志设置为 `true`。

1.3 词法单元识别

（1）判断字符类型：根据当前读取的字符类型，识别不同的词法单元。常见的字符类型包括字母、数字、运算符、分隔符等。

（2）识别关键字和标识符：如果当前字符是字母，则可能是关键字或标识符。通过不断读取后续的字母和数字，组成一个完整的字符串，然后判断该字符串是否为关键字。代码中使用 `isKeyword()` 函数来判断一个字符串是否为关键字。

（3）识别数字：如果当前字符是数字，则可能是整数或浮点数。根据数字的组成规则，判断是整数还是浮点数，并处理可能的错误情况，如以 0 开头的非零整数。

（4）处理其他字符：对于运算符、分隔符等其他字符，直接将其识别为相应的词法单元。

1.4 输出词法单元

（1）创建词法单元对象：识别出一个词法单元后，创建一个 `Token` 对象，包含词法单元的类型、内容、行号和列号等信息。

(2) 返回词法单元：将创建好的 Token 对象返回，作为词法分析的结果。在代码中，getNextToken() 函数负责完成词法单元的识别和返回。

1.5 结束处理

(3) 文件结束标志：当读取到文件末尾时，返回一个表示文件结束的词法单元 (END_OF_FILE)，表示词法分析结束。

2. 在本次实验中完成的简单编译器涉及了哪些在词法分析中的相关操作以及数据结构？

2.1 相关操作

(1) 文件操作：

打开文件：使用 ifstream 打开待分析的源文件，确保能够读取文件内容。在 Lexer 类的构造函数中，使用 file.open(filename) 打开文件。

读取字符：使用 ifstream::get() 方法逐字符读取文件内容，并更新当前的行号和列号。advance() 函数实现了这一操作。

(2) 空白字符和注释处理

跳过空白字符：使用 isspace() 函数判断当前字符是否为空白字符，若为空白字符则继续读取下一个字符，直到遇到非空白字符。skipWhitespace() 函数实现了这一操作。

跳过注释：根据注释的起始符号 (// 或 /*)，判断注释类型，并跳过相应的注释内容。skipComment() 函数实现了这一操作。

(3) 词法单元识别操作

关键字和标识符识别：通过判断字符是否为字母，读取连续的字母和数字组成字符串，然后使用 isKeyword() 函数判断该字符串是否为关键字。

数字识别：根据数字的组成规则，判断是整数还是浮点数，并处理可能的错误情况，如以 0 开头的非零整数。

2.2 相关数据结构

(1) 向量 (vector)

关键字列表：使用 vector<string> 存储编程语言中的关键字，方便进行关键字的查找和判断。在代码中，keywords 向量存储了所有的关键字。

(2) 结构体 (struct)

词法单元：定义一个 Token 结构体，包含词法单元的类型、内容、行号和列号等信息。Token 结构体用于存储识别出的词法单元，并作为词法分析的输出结果。

实验过程及内容：

第一部分：词法分析的理解

1. 代码整体框架已经写好，仅要求填写部分代码或者某些函数实现

(1) Lexer::Lexer(const string& filename):

```
6  Lexer::Lexer(const string &filename) : line(1), col(0), eof(false) {
7  // 初始化关键字列表
8  keywords = {
9      "int", "float", "program", "const", "def",
10     "begin", "end", "call", "let", "if", "then",
11     "else", "fi", "while", "do", "endwh", "return",
12     "and", "or"
13 };
14
15     file.open(filename);
16     if (!file.is_open()) {
17         cerr << "无法打开文件: " << filename << endl;
18         exit(1);
19     }
20     advance(); // 读取第一个字符
21 }
```

此构造函数的作用是对词法分析器进行初始化。它会把关键字列表初始化，尝试打开指定文件，若文件打开失败则输出错误信息并终止程序，最后读取文件的第一个字符。

(2) void Lexer::advance()

```
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
23 void Lexer::advance() {
24     // 读取字符并更新行号、列号
25     if (file.get(currentChar)) {
26         col++;
27         if (currentChar == '\n') {
28             line++;
29             col = 0;
30         }
31     } else {
32         eof = true;
33     }
34 }
```

该函数的功能是从文件里读取下一个字符，同时更新当前的行号和列号。当读取到文件末尾时，会将 eof 标志设置为 true。

(3) bool Lexer::isKeyword(const string& lexeme)

```
71
72 // 判断是否为关键字
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
73 bool Lexer::isKeyword(const string &lexeme) {
74     for (const string &kw : keywords) {
75         if (kw == lexeme) {
76             return true;
77         }
78     }
79     return false;
80 }
```

该函数用于判断给定的字符串是否为关键字。它会遍历关键字列表，若找到匹配的关键词则返回 true，否则返回 false。

(4) Token Lexer::getNextToken()

①跳过空白 + 跳过连续注释（预处理）

```
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
82 Token Lexer::getNextToken() {
83     Token token;
84     skipWhitespace();
85     while (!eof && currentChar == '/') {
86         char next = file.peek();
87         if (next == '/' || next == '*') {
88             skipComment();
89             skipWhitespace();
90         } else {
91             break;
92         }
93     }
94 }
```

跳过空格、制表符、换行等空白字符，这些不是有效单词。while 循环处理连续注释；如果当前字符是 /，查看下一个字符：// 或 /* → 是注释 → 调用 skipComment() 整块跳过。跳过注释后继续跳过空白不是注释就退出循环，开始识别单词。

②记录单词位置（行号、列号）、文件结束处理

```
95         int tokenStartLine = line;
96         int tokenStartCol = col;
97
98         if (eof) {
99             token.type = END_OF_FILE;
100            token.lexeme = "";
101            return token;
102        }
```

单词开始时，保存当前行号和列号，最终打包进 Token。读到文件末尾，返回结束标记，主循环会停止读取。

③识别关键字/标识符（字母开头）

```
104         // 标识符或关键字
105         if (isalpha(currentChar)) {
106             string lexeme;
107             lexeme.push_back(currentChar);
108             advance();
109             while (!eof && isalnum(currentChar)) {
110                 lexeme.push_back(currentChar);
111                 advance();
112             }
113             if (lexeme.size() > 32) {
114                 token.type = ERROR_TOKEN;
115                 token.lexeme = "Error";
116             } else {
117                 token.type = iskeyword(lexeme) ? KEYWORD : IDENTIFIER;
118                 token.lexeme = lexeme;
119             }
120             token.line = tokenStartLine;
121             token.col = tokenStartCol;
122             return token;
123         }
```

当前字符是字母 → 一定是标识符或关键字，读取连续字母 / 数字，把连续的 [a-zA-Z0-9] 全部读进来，组成一个单词。长度检查：长度 >32 → 非法标识符 → 标记错误。区分关键字 vs 标识符在关键字列表里 → KEYWORD、不在 → IDENTIFIER，填充位置信息并返回。

④处理以 0 开头的数字

```

125 // 数字：整数或小数
126 else if (isdigit(currentChar)) {
127     string lexeme;
128
129     if (currentChar == '0') {
130         lexeme.push_back(currentChar);
131         advance();
132
133         // 0 开头跟数字 → 错误
134         if (!eof && isdigit(currentChar)) {
135             while (!eof && isdigit(currentChar)) {
136                 lexeme.push_back(currentChar);
137                 advance();
138             }
139             token.type = ERROR_TOKEN;
140             token.lexeme = "Error";
141             token.line = tokenStartLine;
142             token.col = tokenStartCol;
143             return token;
144         }
145     }
146
147     // 0 后是小数点
148     if (!eof && currentChar == '.') {
149         lexeme.push_back(currentChar);
150         advance();
151         // 小数点后无数字 → 错误
152         if (eof || !isdigit(currentChar)) {
153             token.type = ERROR_TOKEN;
154             token.lexeme = "Error";
155             token.line = tokenStartLine;
156             token.col = tokenStartCol;
157             return token;
158         }
159         // 读取小数部分
160         while (!eof && isdigit(currentChar)) {
161             lexeme.push_back(currentChar);
162             advance();
163         }
164         // 合法浮点数
165         token.type = FLOAT;
166         token.lexeme = lexeme;
167         token.line = tokenStartLine;
168         token.col = tokenStartCol;
169         return token;
170     }
171     // 单纯 0 → 合法整数
172     else {
173         token.type = INTEGER;
174         token.lexeme = lexeme;
175         token.line = tokenStartLine;
176         token.col = tokenStartCol;
177         return token;
178     }
}

```

当读到数字 0 时，先把这个 0 记下来，再看下一个字符是什么。如果后面还跟着别的数字，比如 012 这种，就属于不合法的写法，直接标记成错误；如果后面是小数点，就继续往后读，要是小数点后面没有数字，比如 0.，也算错误，要是数字，比如 0.5，就判定为合法的浮点数；如果 0 后面既不是数字也不是小数点，就说明这只是单独一个 0，判定为合法的整数。

⑤处理非 0 开头的数字

```
179 // 非 0 开头数字
180 else {
181     while (!eof && isdigit(currentChar)) {
182         lexeme.push_back(currentChar);
183         advance();
184     }
185     // 有小数点
186     if (!eof && currentChar == '.') {
187         lexeme.push_back(currentChar);
188         advance();
189         // 小数点后无数字 → 错误
190         if (eof || !isdigit(currentChar)) {
191             token.type = ERROR_TOKEN;
192             token.lexeme = "Error";
193             token.line = tokenStartLine;
194             token.col = tokenStartCol;
195             return token;
196         }
197         // 读取小数部分
198         while (!eof && isdigit(currentChar)) {
199             lexeme.push_back(currentChar);
200             advance();
201         }
202         // 合法浮点数
203         token.type = FLOAT;
204         token.lexeme = lexeme;
205         token.line = tokenStartLine;
206         token.col = tokenStartCol;
207         return token;
208     }
209     // 无小数点 → 合法整数
210     else {
211         token.type = INTEGER;
212         token.lexeme = lexeme;
213         token.line = tokenStartLine;
214         token.col = tokenStartCol;
215         return token;
216     }
217 }
218 }
```

当读到 1-9 开头的数字时，先把连续的数字都读完，形成整数部分。读完后如果遇到小数点，就再往后看，要是小数点后面没有数字，比如 123.，就标记成错误；要是后面有数字，就把小数部分也读完，判定为合法的浮点数；如果读完整数后没有小数点，就直接判定为合法的整数。

⑥处理分隔符、运算符及其他符号

```
220 // 分隔符、运算符及其他符号
221 else {
222     char curr = currentChar;
223     if (curr == '(' || curr == ')' || curr == ';' || curr == ',') {
224         token.type = DELIMITER;
225         token.lexeme = string(1, curr);
226         token.line = tokenStartLine;
227         token.col = tokenStartCol;
228         advance();
229         return token;
230     }
231     if (curr == '+' || curr == '-' || curr == '*') {
232         token.type = ARITH_OP;
233         token.lexeme = string(1, curr);
234         token.line = tokenStartLine;
235         token.col = tokenStartCol;
236         advance();
237         return token;
238     }
```



```

239     if (curr == '/') {
240         char next = file.peek();
241         if (next != '/' && next != '*') {
242             token.type = ARITH_OP;
243             token.lexeme = string(1, curr);
244             token.line = tokenStartLine;
245             token.col = tokenStartCol;
246             advance();
247             return token;
248         } else {
249             skipComment();
250             return getNextToken();
251         }
252     }
253     if (curr == '=') {
254         advance();
255         if (!eof && currentChar == '=') {
256             token.type = COMP_OP;
257             token.lexeme = "==";
258             token.line = tokenStartLine;
259             token.col = tokenStartCol;
260             advance();
261             return token;
262         } else {
263             token.type = ASSIGN_OP;
264             token.lexeme = "=";
265             token.line = tokenStartLine;
266             token.col = tokenStartCol;
267             return token;
268         }
269     }

270     if (curr == '<') {
271         advance();
272         if (!eof && currentChar == '=') {
273             token.type = COMP_OP;
274             token.lexeme = "<=";
275             token.line = tokenStartLine;
276             token.col = tokenStartCol;
277             advance();
278             return token;
279         } else if (!eof && currentChar == '>') {
280             token.type = COMP_OP;
281             token.lexeme = "<>";
282             token.line = tokenStartLine;
283             token.col = tokenStartCol;
284             advance();
285             return token;
286         } else {
287             token.type = COMP_OP;
288             token.lexeme = "<";
289             token.line = tokenStartLine;
290             token.col = tokenStartCol;
291             return token;
292         }
293     }

```

```

294  ✓      if (curr == '>') {
295          advance();
296  ✓      if (!eof && currentChar == '=') {
297          token.type = COMP_OP;
298          token.lexeme = ">=";
299          token.line = tokenStartLine;
300          token.col = tokenStartCol;
301          advance();
302          return token;
303  ✓      } else {
304          token.type = COMP_OP;
305          token.lexeme = ">";
306          token.line = tokenStartLine;
307          token.col = tokenStartCol;
308          return token;
309      }
310  }
311  // 无法识别的字符视为错误
312  token.type = ERROR_TOKEN;
313  token.lexeme = string(1, curr);
314  token.line = tokenStartLine;
315  token.col = tokenStartCol;
316  advance();
317  return token;
318  }
319  }

```

对于既不是字母也不是数字的符号，会按类型逐一判断：像()_;这类符号直接当作分隔符；+ - * /直接当作算术运算符，其中/要先判断是不是注释，是注释就跳过，不是才当作除号；=单独出现是赋值符号，跟后面的=连在一起==就是比较符号；< >以及组合出来的<= >= <>都当作比较符号；不在这些规则里的奇怪符号，全都当成错误符号处理。

2.测试最终代码

(1) 使用 test.txt，运行结果：

```

● → /workspace git:(master) X ./lex examples/test.txt
(1, int, 1, 1)
(2, a, 1, 6)
(5, ;, 1, 7)
(1, let, 2, 1)
(2, a, 2, 5)
(8, =, 2, 7)
(3, 30, 2, 9)
(5, ;, 2, 11)

```

(2) 使用 source_correct.txt, 运行结果:

```
● → /workspace git:(master) X ./lex examples/source_correct.txt
(1, int, 1, 1)
(2, a, 1, 5)
(5, ;, 1, 6)
(1, let, 2, 1)
(2, a, 2, 5)
(8, =, 2, 7)
(3, 30, 2, 9)
(5, ;, 2, 11)
(1, float, 3, 1)
(2, b, 3, 7)
(8, =, 3, 9)
(4, 3.14, 3, 11)
(5, ;, 3, 15)
(1, if, 4, 1)
(2, a, 4, 4)
(7, <, 4, 6)
(3, 10, 4, 8)
(1, then, 4, 11)
(2, a, 5, 5)
(8, =, 5, 7)
(2, a, 5, 9)
(6, +, 5, 11)
(3, 1, 5, 13)
(5, ;, 5, 14)
(1, else, 6, 1)
(2, a, 7, 5)
(8, =, 7, 7)
(2, a, 7, 9)
(6, -, 7, 11)
(3, 1, 7, 13)
(5, ;, 7, 14)
(1, fi, 8, 1)
(1, program, 12, 1)
(2, main, 12, 9)
(5, ;, 12, 13)
```

(3) 使用 source_error.txt, 运行结果:

```
● → /workspace git:(master) X ./lex examples/source_error.txt
(1, int, 1, 1)
(-1, Error, 1, 5)
(5, ;, 1, 9)
(1, float, 2, 1)
(2, num, 2, 7)
(8, =, 2, 11)
(-1, Error, 2, 13)
(5, ;, 2, 15)
(1, let, 3, 1)
(-1, Error, 3, 5)
(8, =, 3, 39)
(3, 100, 3, 41)
(5, ;, 3, 44)
(1, if, 4, 1)
(2, num, 4, 4)
(7, >=, 4, 8)
(3, 50, 4, 11)
(1, then, 4, 14)
(2, a, 4, 19)
(8, =, 4, 21)
(2, a, 4, 23)
(6, +, 4, 25)
(3, 1, 4, 27)
(5, ;, 4, 28)
```

第二部分：程序的测试

在完成第一部分后，写一点简单的代码，输入到上面的词法分析器中，并给出执行词法分析后的输出。

(1) 使用 test1.txt (复杂变量定义与运算)

代码：

```
● → /workspace git:(master) X cat test1.txt
float price;
let price = 99.99;
let total = price * 2;
let avg = total / 3;
```

运行结果：

```
● → /workspace git:(master) X ./lex test1.txt
(1, float, 1, 1)
(2, price, 1, 7)
(5, ;, 1, 12)
(1, let, 2, 1)
(2, price, 2, 5)
(8, =, 2, 11)
(4, 99.99, 2, 13)
(5, ;, 2, 18)
(1, let, 3, 1)
(2, total, 3, 5)
(8, =, 3, 11)
(2, price, 3, 13)
(6, *, 3, 19)
(3, 2, 3, 21)
(5, ;, 3, 22)
(1, let, 4, 1)
(2, avg, 4, 5)
(8, =, 4, 9)
(2, total, 4, 11)
(6, /, 4, 17)
(3, 3, 4, 19)
(5, ;, 4, 20)
```

(2) 使用 test2.txt (包含注释和空白的代码)

代码：

```
● → /workspace git:(master) X cat test2.txt
// 定义整数变量
int count;

/*
多行注释
测试
*/
let count = 5;
```

运行结果：

```
● → /workspace git:(master) X ./lex test2.txt
(1, int, 2, 1)
(2, count, 2, 5)
(5, ;, 2, 10)
(1, let, 8, 1)
(2, count, 8, 5)
(8, =, 8, 11)
(3, 5, 8, 13)
(5, ;, 8, 14)
```

(2) 使用 test3.txt (条件判断语句)

代码:

```
● → /workspace git:(master) X cat test3.txt
if (count > 0) then
    let result = 1;
else
    let result = 0;
fi
```

运行结果:

```
● → /workspace git:(master) X ./lex test3.txt
(1, if, 1, 1)
(5, (, 1, 4)
(2, count, 1, 5)
(7, >, 1, 11)
(3, 0, 1, 13)
(5, ), 1, 14)
(1, then, 1, 16)
(1, let, 2, 5)
(2, result, 2, 9)
(8, =, 2, 16)
(3, 1, 2, 18)
(5, ;, 2, 19)
(1, else, 3, 1)
(1, let, 4, 5)
(2, result, 4, 9)
(8, =, 4, 16)
(3, 0, 4, 18)
(5, ;, 4, 19)
(1, fi, 5, 1)
```

实验结论:

本次编译原理词法分析程序设计实验已顺利完成,我按照实验要求成功实现了一个具备完整词法分析能力的词法分析器。该分析器能够正确识别关键字、标识符、整数、浮点数、分隔符、四则运算符、比较运算符和赋值运算符等八类单词,同时支持空白字符跳过、单行注释与多行注释处理,并能对非法数字、超长标识符、无效符号等错误进行准确标记。通过对 test.txt、source_correct.txt、source_error.txt 及三组自定义测试用例的运行验证,程序输出的单词类型、词素、行号、列号均符合规范,整体逻辑正确、运行稳定,达到了实验预设的全部目标。本次实验有效验证了词法分析的核心流程,加深了我对编译工作原理的理解。

心得体会:

通过本次词法分析实验,我不仅完成了代码补全与调试,更从实践层面真正理解了词法分析在编译过程中的重要作用。词法分析作为编译器的第一道工序,负责将无结构的字符流转化为有意义的单词序列,看似简单,实则对规则判断、边界处理、错误识别都有很高要求。在实现过程中,我深刻体会到细节决定正确性。例如数字识别中以 0 开头的整数、小数点后无数字的浮点数、标识符长度限制等,都需要严格按照规则判断;运算符的区分,如赋值号 “=” 与比较符 “==”、除号 “/” 与注释开头等,也需要仔细处理,否则会出现识别错误。同时,行号与列号的准确更新、空白与注释的完整跳过,都是保证输出结果正确的

关键。

通过补全 `Lexer` 类的构造函数、`advance()`、`isKeyword()` 以及核心函数 `getNextToken()`，我熟悉了 C++ 文件操作、字符串处理、向量与结构体的使用，也理解了词法分析器的整体架构。从一开始对框架的陌生，到逐步理清预处理、识别、返回、错误处理的完整流程，我逐渐建立起对编译器工作方式的直观认识。

本次实验让我把抽象的词法规则、状态转移、Token 等概念转化为可运行、可调试的真实程序，不仅提升了编程与调试能力，也为后续语法分析、语义分析等更复杂的编译原理内容打下了坚实基础。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注: